Parcial 2 — Compresor Huffman concurrente integrado al editor

Autor: Juan Fernando Romero · **Asignatura:** Sistemas Operativos — SO2026B (EAFIT)
Alternativa: 1 — Compresor Huffman concurrente · **Tamaño de equipo:** 1 (individual)

El editor del Parcial 1 (`edit` dentro del shell `eafitOS`) ganó cuatro comandos: `z` (comprimir), `x` (descomprimir), `s` (progreso) y `w` (esperar). La compresión corre en segundo plano con un pool de 4 hilos, y el editor sigue respondiendo mientras tanto.

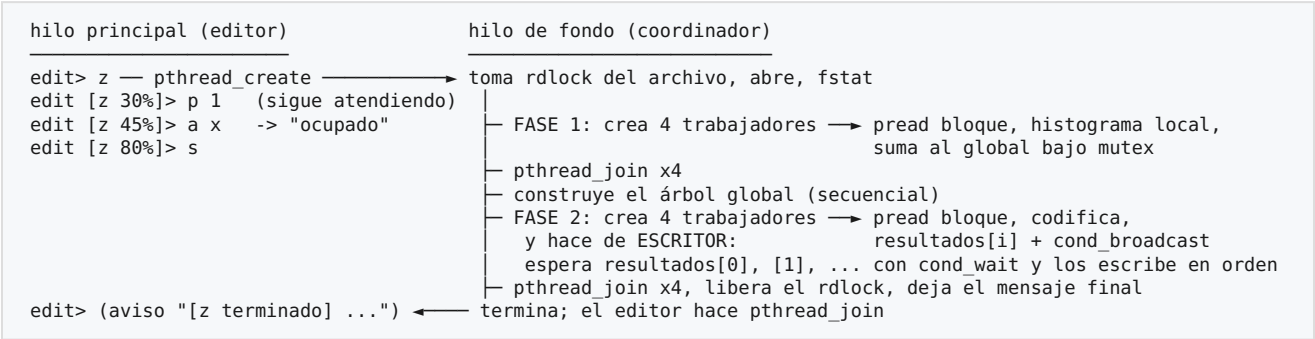
Archivos

Archivo	Qué contiene
`shell/huffman.c/.h`	Huffman sin hilos: árbol a partir de las frecuencias, tabla de códigos, codificar y decodificar un bloque.
`shell/compresor.c/.h`	Los hilos: conteo paralelo, codificación paralela, escritura en orden, descompresión, estado del progreso.
`shell/editor.c`	Comandos `z/x/s/w`, prompt con progreso, rwlock en `p/a/d`, Ctrl+C.
`shell/test_parcial2.sh`	Pruebas: integridad (md5/cmp), demo de no bloqueo, valgrind y helgrind.
`shell/evidencias/`	Salida de valgrind/helgrind y md5 de los archivos de prueba.

Compilar y probar: `cd shell && make && ./test_parcial2.sh` (`make run` abre el shell).

---

1. Concurrencia en Huffman (25 pts)



- **Bloques:** el archivo se parte en bloques de 64 KiB. Los trabajadores toman el siguiente bloque libre de un contador compartido (`siguiente`) protegido por mutex. Así el reparto es dinámico: un hilo que termina antes toma otro bloque.
- **Fase 1 (frecuencias), con reducción local:** cada hilo lee su bloque con `pread` (lleva su propio offset, así que no comparte el offset del descriptor) y cuenta en un histograma local `uint64_t local[256]`. Al terminar el bloque, suma el histograma al global bajo el mutex. Toma el mutex una vez por bloque y no una vez por byte.
- **Árbol global:** después del `pthread_join` de la fase 1 se construye un solo árbol, de forma secuencial. Construirlo cuesta poco (máximo 256 hojas).

- **Fase 2 (codificación):** 4 hilos codifican bloques en paralelo usando el árbol, que en esta fase solo se lee y por eso no necesita lock.
- **Salida en orden:** cada trabajador deja su bloque en `resultados[i]`, marca `listo = 1` y hace `pthread_cond_broadcast`. El coordinador escribe el bloque 0, luego el 1, y así sucesivamente: para cada `i` espera con `pthread_cond_wait` hasta que `resultados[i].listo`, lo escribe con `write` y libera su memoria. Los bloques pueden terminar en cualquier orden, pero siempre se escriben en orden.
- **Formato `.huf`:** `"HUF1"` | tamaño original | nº de bloques | 256 frecuencias, y por cada bloque `[bytes originales][bytes comprimidos][datos]`. El descompresor reconstruye el mismo árbol a partir de las frecuencias, porque la construcción es determinista.
- **Descompresión:** es secuencial, porque el enunciado solo pide que no bloquee el editor. Corre en el mismo hilo de fondo que la compresión.

2. Mecanismos de sincronización (25 pts)

Recurso compartido	Quién lo usa	Qué lo protege
`siguiente` (próximo bloque libre)	trabajadores	`c->mutex`
`freq[256]` global	trabajadores de la fase 1	`c->mutex` (una suma por bloque)
`resultados[i]`, `error`	trabajadores de la fase 2 ↔ escritor	`c->mutex` + `cond_listo` (variable de condición)
Árbol de Huffman	trabajadores de la fase 2	No necesita lock: se construye antes de crearlos y luego solo se lee
Progreso (`hechos/total`), `activo`, mensaje final	hilo de fondo ↔ editor (`s`, prompt)	`g_estado.mutex`
Arranque del hilo de fondo	editor ↔ hilo de fondo	`g_estado.mutex` + `cond_arranque`
El archivo abierto en el editor	compresor y `p` (lectores) / `a` y `d` (escritores)	`pthread_rwlock_t g_archivo_lock`
Offset del descriptor de entrada	trabajadores	Ninguno: cada hilo usa `pread` con su propio offset

Sin espera activa. Todas las esperas duermen al hilo: el escritor usa `pthread_cond_wait` sobre `cond_listo`, `z` espera el arranque con `pthread_cond_wait` sobre `cond_arranque`, y para esperar el final se usa `pthread_join` (en `w`, en `q` y entre fases). No hay ningún `while(!listo);` ni `usleep`. Cada `cond_wait` está dentro de un `while` que revisa la condición, por si hay despertares espurios.

Sin interbloqueos (deadlock).

- Ningún hilo tiene dos mutex a la vez: `c->mutex` y `g_estado.mutex` nunca se anidan, así que no puede haber espera circular.
- El único lock que se sostiene mucho tiempo es el rwlock de lectura del hilo de compresión. El editor nunca se queda esperando ese lock: `a`/`d` usan `pthread_rwlock_trywrlock`, que falla de inmediato con `EBUSY`, y `p` pide lectura, que es compatible con otro lector.
- Si un trabajador falla, `marcar_error` hace `broadcast`. El escritor revisa `error` en la misma condición y deja de esperar un bloque que ya nadie va a producir. Por la misma razón, si no se pudo crear ningún trabajador se marca error.

Sin condiciones de carrera. Todo dato que escriben varios hilos está en la tabla de arriba, protegido por su mutex. El hilo de fondo no lee `g_entrada`/`g_salida` hasta que `pthread_create` lo

arranca, y el editor ya las escribió antes de crearlo (`pthread_create` garantiza ese orden). Helgrind lo confirma (ver Evidencia).

3. Integración con el editor (20 pts)

- `z` y `x` lanzan **un hilo en segundo plano** y regresan al prompt de inmediato.
- El prompt muestra el progreso mientras hay trabajo (`edit [z 45%]>`). El comando `s` muestra la barra `[z] [####......] 45% (73/162 bloques)`.
- Cuando el trabajo termina, el siguiente prompt avisa **una vez** (`[z terminado] Comprimido ...`). Los hilos de fondo **nunca imprimen**, para no mezclar su salida con lo que escribe el usuario: dejan el resultado o el error en `g_estado.mensaje` y el editor lo imprime.
- Solo se permite un trabajo a la vez: `z`/`x` se rechazan si ya hay uno activo, y `o` (abrir otro archivo) también se rechaza.
- **Editar mientras se comprime** (la condición de carrera que pide el enunciado):
  - El hilo de compresión toma el **lock de lectura** durante toda la compresión.
  - `p` también toma lectura, así que puede mostrar el archivo **al mismo tiempo**.
  - `a` y `d` necesitan **escritura** y usan `pthread_rwlock_trywrlock`. Si el lock está ocupado muestran `Archivo ocupado: compresión en curso (45%)` **sin congelar el editor**.
  - Sin esto, `d` podría acortar el archivo mientras un trabajador hace `pread`: el `.huf` mezclaría la versión vieja con la nueva y quedaría corrupto.
  - `z` no regresa hasta que el hilo de fondo tiene el lock de lectura. Esa espera dura microsegundos y se hace con una variable de condición. Así ninguna edición se puede colar entre `z` y el inicio de la lectura.

Decisión simple: se rechaza la edición en vez de encolarla o copiar el archivo a memoria. Es lo más fácil de explicar y cumple la rúbrica: el usuario puede seguir usando el editor y nunca se corrompe ni el archivo ni el `.huf`.

4. Corrección algorítmica (15 pts)

`test_parcial2.sh` comprime y descomprime cada caso **a través del editor** (`z`, `w`, `x`, `w` por stdin) y compara con `cmp` y `md5sum`:

Caso	Por qué es interesante
Archivo vacío (0 B)	0 bloques, sin árbol.
1 byte	Un bloque mínimo y un solo símbolo.
Un solo carácter repetido (200 KB de `a`)	El árbol es solo una hoja: se le asigna el código de 1 bit `"0"`.
Binario aleatorio (1 MB)	Los 256 símbolos aparecen; casi no se comprime.
Tamaño no múltiplo de 64 KiB (~135 KB)	El último bloque es más corto.
Texto mediano (~1,2 MB) y grande (~10 MB)	Muchos bloques: prueba el orden de salida con 4 hilos.

5. Manejo de errores y recursos (15 pts)

- **Memoria:** cada bloque comprimido se libera apenas se escribe. Al final (también en caso de error) se libera lo que quede, mediante un único bloque de limpieza (`goto fin`).
- **Descriptores:** cada `open` tiene su `close` en ese mismo bloque de limpieza. Si `close` falla, también se reporta el error.

- **Hilos:** cada `pthread_create` tiene su `pthread_join`. Entre fases se hace join de los trabajadores. El editor hace join del hilo de fondo en `w`, en el prompt siguiente al final del trabajo o antes de iniciar otro. `q` **espera con join** a que termine el trabajo antes de salir, así que nunca quedan hilos vivos.
- **Errores de syscalls:** el hilo principal sigue el patrón del Parcial 1 (`perror` / `LOG_SYSCALL_ERROR`). Los hilos guardan `strerror(errno)` en el estado y el editor lo muestra en rojo. Si un trabajador falla, el error detiene a los demás. `.huf` inválidos o corruptos se detectan (cabecera, tamaños, bits insuficientes) sin crash.
- **Señales:** antes de `pthread_create` se bloquea `SIGINT` con `pthread_sigmask`. El hilo de fondo y los trabajadores heredan esa máscara, así que **Ctrl+C solo lo atiende el hilo principal**. En el editor, su manejador (que solo usa `write`, que es async-signal-safe) no mata el proceso: indica que se use `q`, que espera a que termine el trabajo. Así un Ctrl+C no deja un `.huf` escrito a medias ni hilos sueltos. `SA_RESTART` evita que `fgets` se interrumpa.

Evidencia

Integridad (md5)

```
7fc56270e7a70fa81a5935b72eacbe29  prueba_1byte.txt
7fc56270e7a70fa81a5935b72eacbe29  prueba_1byte.txt.out
17cfbd7d9683ae176d777479cf12f947  prueba_binario.bin
17cfbd7d9683ae176d777479cf12f947  prueba_binario.bin.out
d79fae1ed6132bc46614ec46d9446990  prueba_grande.txt
d79fae1ed6132bc46614ec46d9446990  prueba_grande.txt.out
9ee0f33e1a81d5ce377a64263c9c4e4f  prueba_mediano.txt
9ee0f33e1a81d5ce377a64263c9c4e4f  prueba_mediano.txt.out
49b6e203695e8d05c491c74175ff52d7  prueba_no_multiplo.txt
49b6e203695e8d05c491c74175ff52d7  prueba_no_multiplo.txt.out
561b1994f6baacd6e5eaf4baaa12849f  prueba_repetido.txt
561b1994f6baacd6e5eaf4baaa12849f  prueba_repetido.txt.out
d41d8cd98f00b204e9800998ecf8427e  prueba_vacio.txt
d41d8cd98f00b204e9800998ecf8427e  prueba_vacio.txt.out
```

Cada original y su `.out` tienen el mismo md5, y `cmp` no reporta diferencias.

Valgrind — memoria y descriptores (`evidencias/valgrind_memcheck.txt`)

```
FILE DESCRIPTORS: 3 open (3 inherited) at exit.
    in use at exit: 0 bytes in 0 blocks
  total heap usage: 75 allocs, 75 frees, 3,300,852 bytes allocated
All heap blocks were freed -- no leaks are possible
ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Helgrind — condiciones de carrera (`evidencias/valgrind_helgrind.txt`)

```
ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 884 from 89)
```

- Memcheck: 0 bytes en uso al salir y 0 errores. Los 3 descriptores abiertos son stdin/stdout/stderr, heredados de la terminal: todos los que abre el programa se cierran.
- Helgrind: 0 errores (sin carreras de datos ni inversión del orden de locks). Los “suppressed” son las supresiones por defecto de valgrind para el código interno de glibc, no errores del programa.
- Las corridas de valgrind usan `prueba_mediano.txt` (~1,2 MB, 19 bloques), comprimido y descomprimido con 4 hilos trabajadores.